Student Services Platform – Appointment & Notification System

Project Title: Student Services Platform – Appointment & Notification System

Course: CS378 – introduction to Web Development

Section: 471

Instructor: Dr. Kajal

Submitted by:

Wafa [redacted] – Front-End Developer

Sarah [redacted] – Back-End Developer

Amjad Mohammed 441500413 – Database Designer

Submission Date: 12/6/2026

Table of Contents

1. Project Proposal

2. Requirements Analysis

3. System Design

4. Implementation

5. Testing & Validation

6. Project Progress

7. Final Tool Description

8. Conclusion

9. References

10. Appendices

1.Project Proposal

1.1 Introduction / Background:

The Student Services Platform — Appointment & Notification System is a web-based application designed to simplify student–staff communication. It allows students to search for instructors, view profiles, book appointments, and receive notifications.

The system also implements session-based authentication to secure access and cookies to provide a personalized login experience.

1.2 Project Scope:

- Provide students with a searchable directory of instructors.

- Allow students to book appointments based on available office hours.

- Display appointment details and statuses.

- Provide subscription-based notifications for office hour updates or cancellations.

- Secure login system for student access.

- Improve usability by remembering student email using cookies.

1.3 Objectives:

- Improve communication between students and instructors.

- Develop an intuitive, responsive user interface.

- Integrate a reliable MySQL database with full CRUD operations.

- Ensure smooth appointment booking without conflicts.

- Provide a notification system for real-time updates.

- Enhance user experience using cookie-based email recall.

- Protect system access using sessions and authentication logic.

1.4 Expected Outcomes:

- Fully functional student appointment system.

- Searchable staff directory with dynamic filtering.

- Subscription-based notifications.

- Secured session-based login.

- Cookie-enabled auto-filled login email.

- Web application deployed on Tomcat using JSP/Servlet logic.

1.5 Tools and Technologies:

- Front-End: HTML, CSS, JSP

- Back-End: Java, JDBC, Session & Cookie management

- Database: MySQL

- Server: Apache Tomcat

- IDE: Eclipse

2. Requirements Analysis

2.1 Functional Requirements:

- Student login authentication (session-based)

- Auto-fill login email (cookie-based)

- Search for instructors by department or name

- View instructor profiles and office information

- Browse available office hours

- Book appointments

- View appointment status (Confirmed / Pending / Cancelled)

- Subscribe/unsubscribe to notifications

- Logout functionality (session termination)

2.2 Non-Functional Requirements:

-System must be secure (session-based authentication)

-Interface must be responsive and user-friendly

-Database should respond in < 1 second for queries

-System must support concurrent appointment requests

-Cookie handling must follow security rules (non-sensitive storage)

-Error handling for invalid login, missing fields, etc.

2.3 Use Case Description:

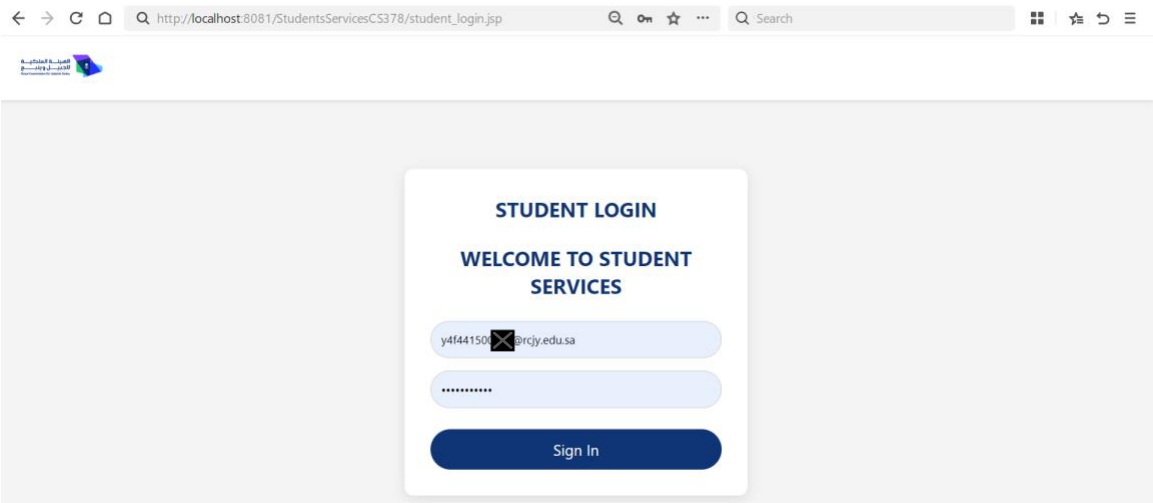
Actor: Student

Use Cases: Login, Search Staff, View Department, View Staff Profile, Book Appointment, View Appointments, Manage Notifications, Logout

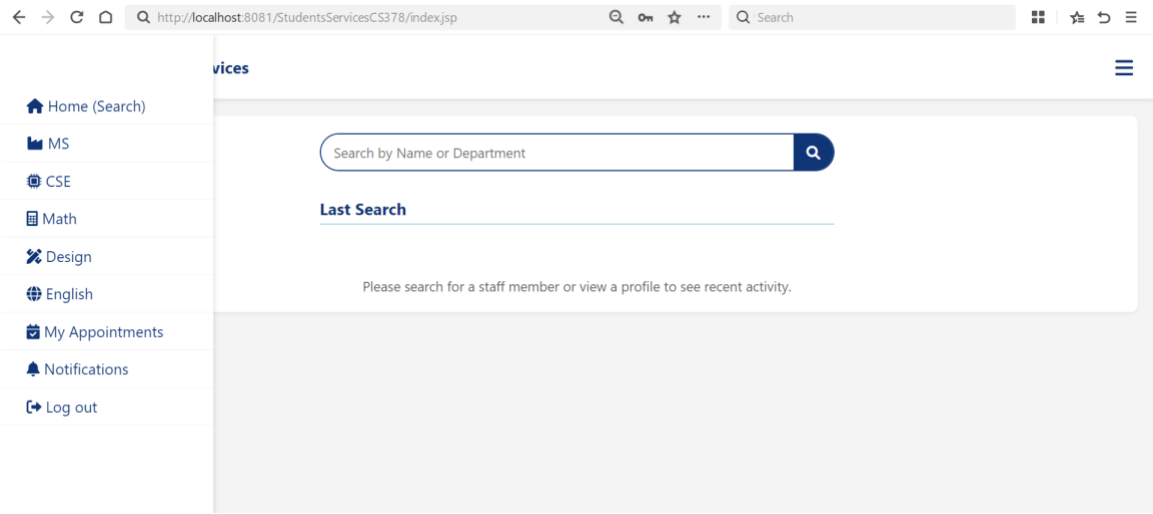
3. System Design

3.1 Front-End Design :

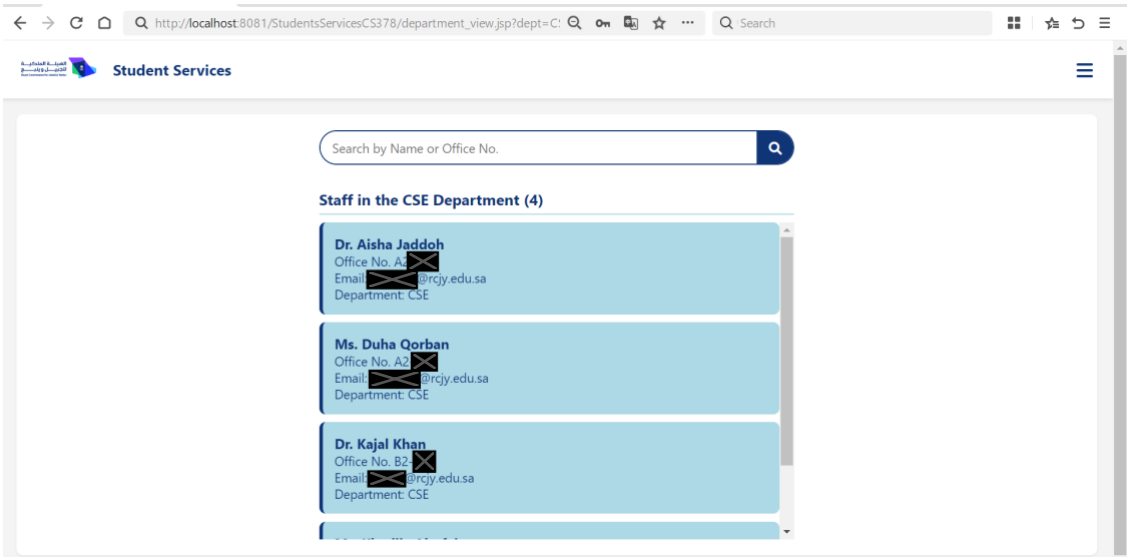
-Login Page (with cookie-based email display)

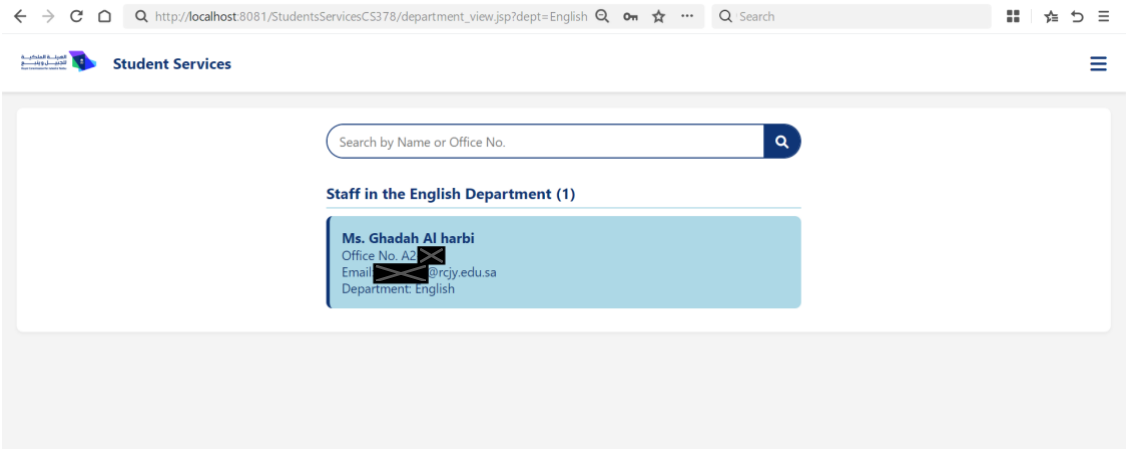


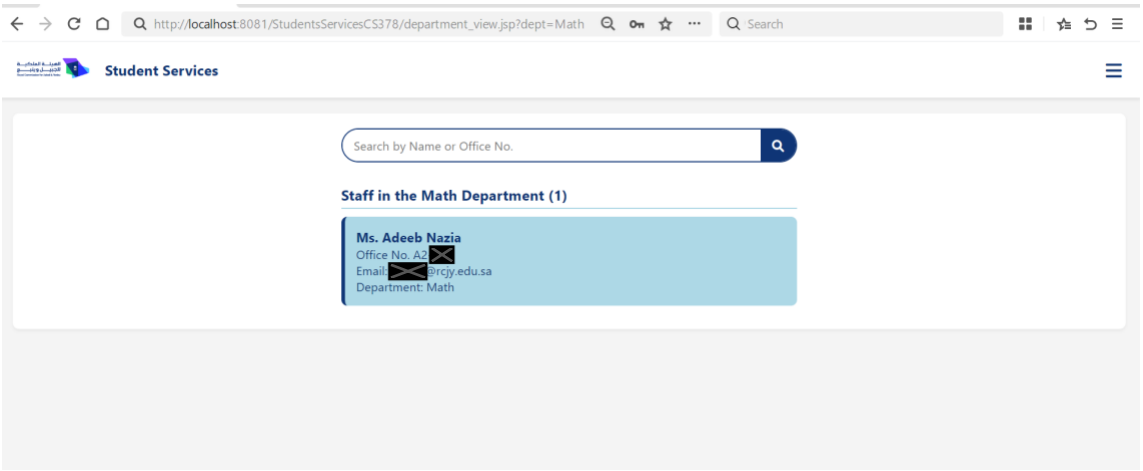
-Home Search Page



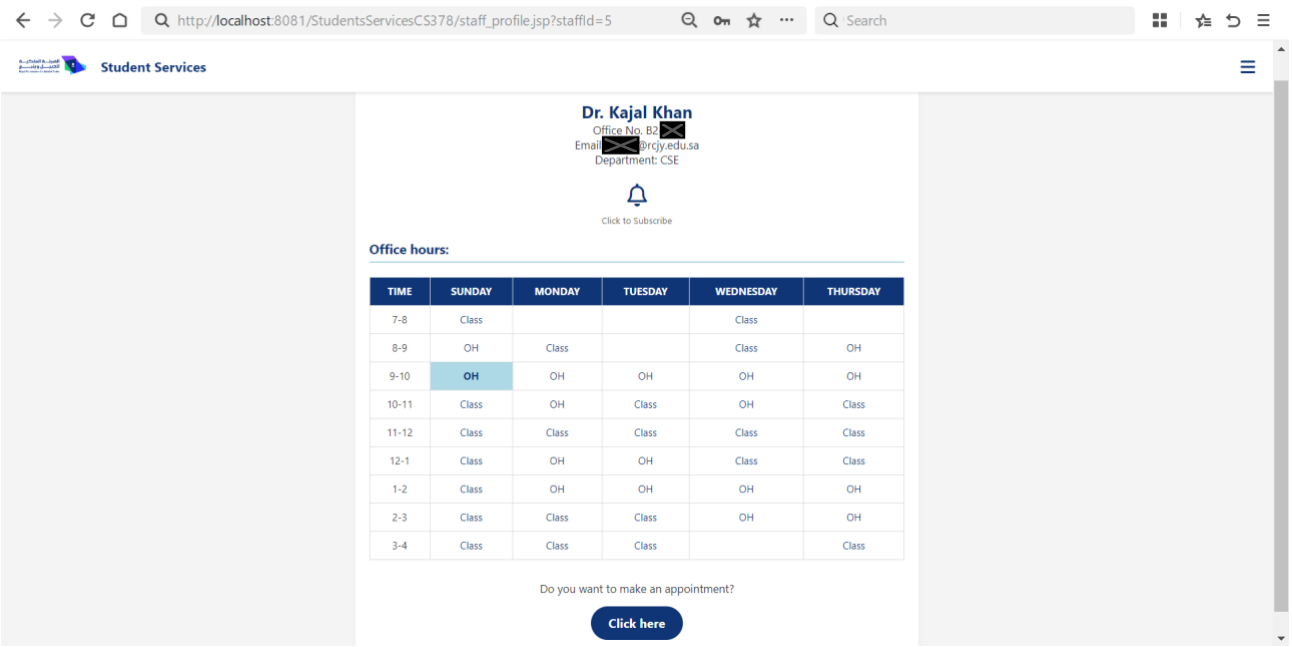
-Departments Pages



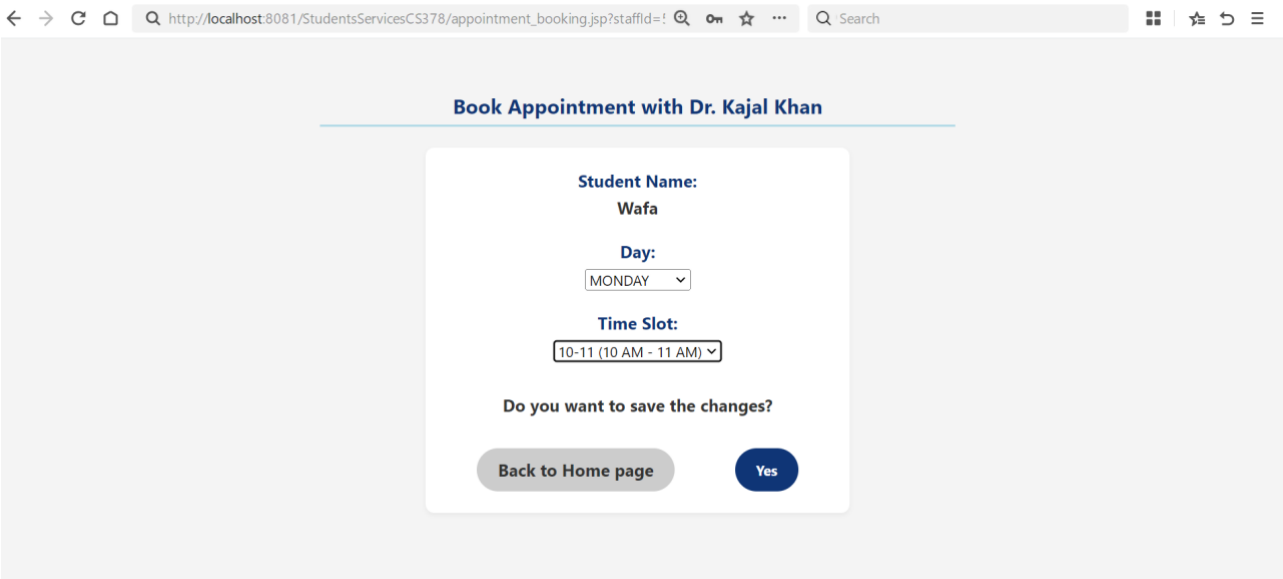


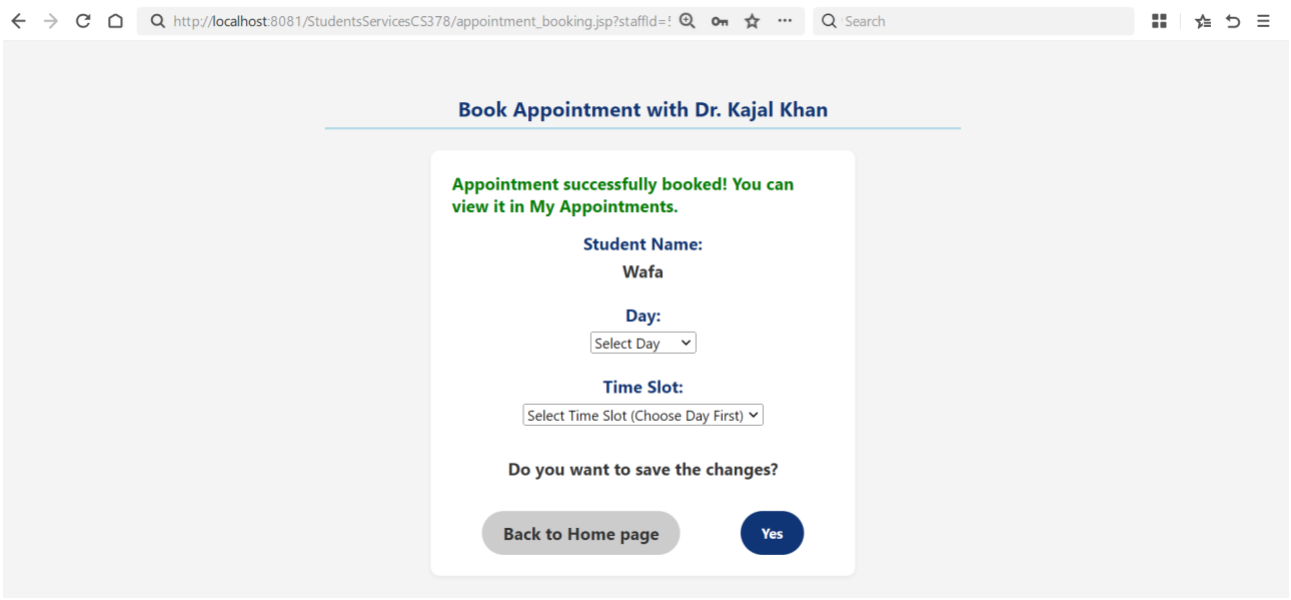


-Staff Profile

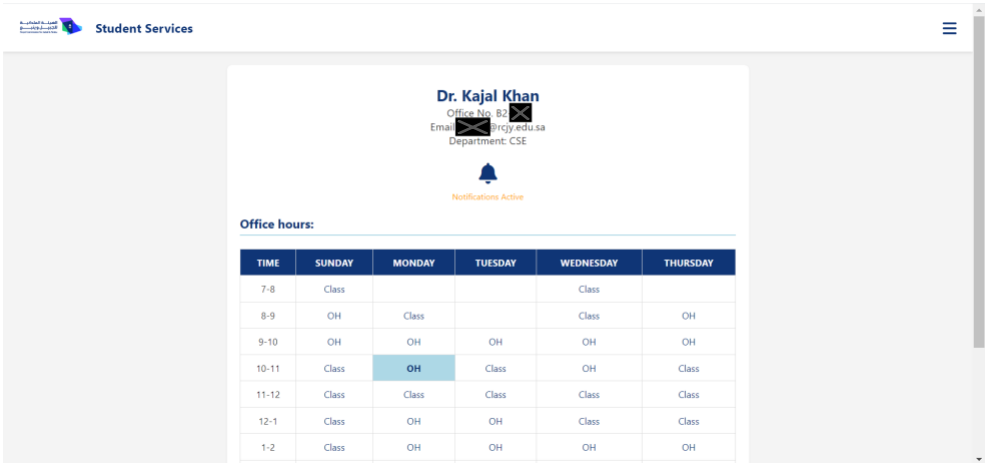


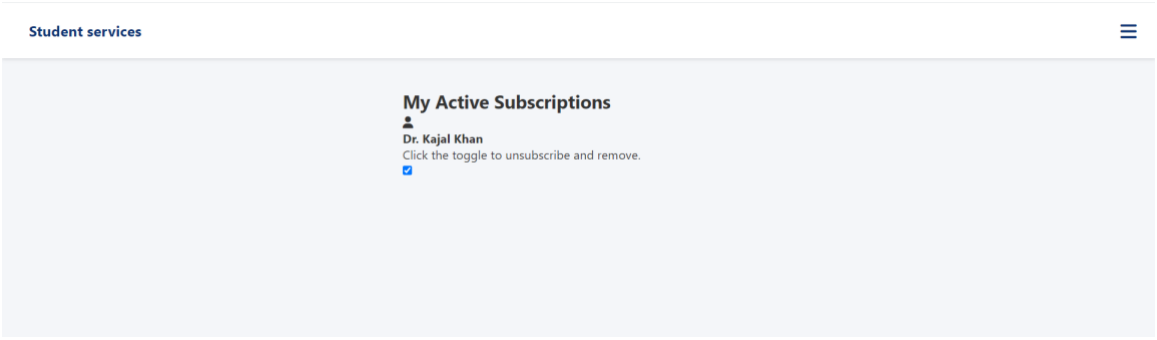
-Appointment Booking Page



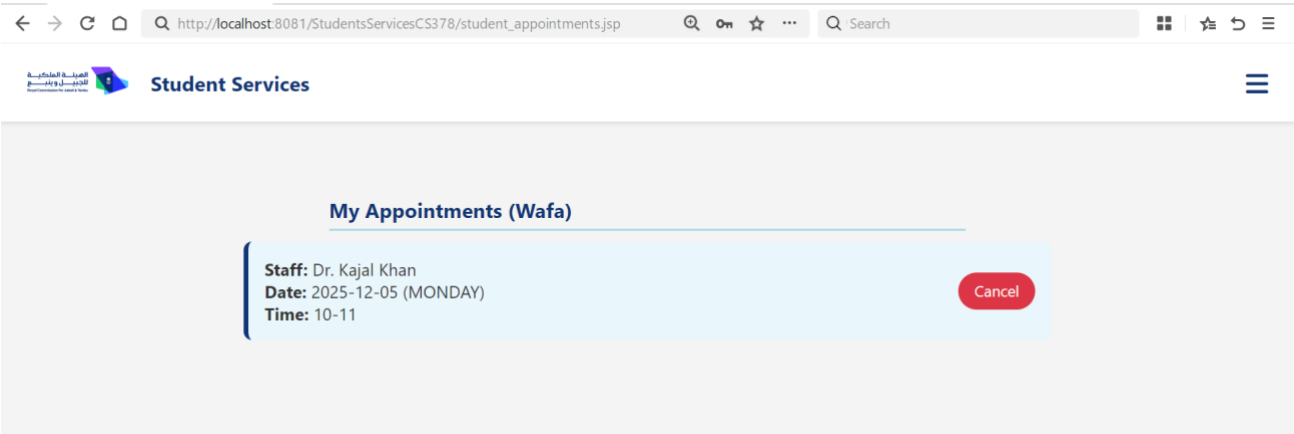


-Notifications Page





-My Appointments Page



3.2 Architecture Overview :

3-tier architecture:

1. Presentation Layer: JSP + CSS
2. Logic Layer: Java (JDBC, sessions, cookies, validation)
3. Database Layer: MySQL (Students, Staff, Appointments, Notifications)

3.3 Database Design :

Core Tables:

- Students (Student_ID, Student_Name, Email, Password_Hash)
- Staff (Staff_ID, Dept_ID, Staff_Name, Email, Office_No)
- Appointments (Appointment_ID, Staff_ID, Student_Name, Email, Date, Time, Status)
- Office_Hours (Staff_ID, Day, Time_Range)
- Notifications (Email, Staff_ID, Message, Date)

3.4 Technology Stack :

- JSP / Java
- Tomcat Server
- MySQL
- JDBC Driver
- CSS Flex/Grid Layout
- HTTP Sessions & Cookies

4. Implementation

4.1 Front-End Implementation

- Designed using JSP with embedded Java logic
- CSS for layout, sidebar, navigation, responsive design
- FontAwesome for icons
- Login form with email pre-filled using cookie
- Search bar + staff results list
- Appointment booking UI
- Notification display UI

4.2 Back-End Implementation

- DBConnection.java for MySQL connection
- SQL queries using PreparedStatement
- Sessions to store student email/name after login
- Cookies to remember student email for one week
- Query parameters for staffId, dept, search inputs

4.3 Database Interaction (CRUD)

- Create: Book appointments
- Read: Staff list, search results, notifications
- Update: Appointment status
- Delete: Cancel subscriptions

4.4 Security Features

- Session authentication
- Redirect unauthorized users to login page
- Cookie auto-fill for non-sensitive data only
- Input validation on login fields
- Session invalidation on logout

5. Testing & Validation

5.1 Test Cases:

Scenario	Expected Result	Status
Valid login	Redirect to home page	Passed
Invalid login	Show error message	Passed
Cookie loaded	Email auto filled	Passed
Access protected page without session	Redirect to login	Passed
Book an appointment	Stored in DB	Passed
Cancel subscription	Removed from DB	Passed

5.2 Bug list & Fixes :

Bug 1: Login page always reloaded without logging in

- Description: When the student entered a correct email and password, the page refreshed but did not redirect to the main page.
- Cause: The SQL query was using the wrong column name for the password field.
- Fix: Updated the query to use the correct column Password_Hash and verified the mapping with the Students table.

Bug 2: Protected pages accessible without login

- Description: Some pages such as the main page and department view could be opened directly by typing the URL, even without logging in.
- Cause: Missing session validation at the top of those JSP pages.
- Fix: Added a session check:

```
if (session.getAttribute("studentEmail") == null) {
response.sendRedirect("student_login.jsp");
  return;
}
```

Bug 3: Notifications page not showing any data

- Description: The notifications/active subscriptions page loaded with an empty list, even when the student was subscribed.
- Cause: The SQL query was using a wrong or missing filter on the student's email.
- Fix: Updated the query to filter by the email from the session and tested with different users.

5.3 User Feedback :

- UI is clean and simple
- Appointment booking is easy to use
- Auto-filled login email is convenient

6. Project Progress

6.1 Completed Features:

- Login
- Search
- Department pages
- Staff profile
- Appointment booking
- Notifications
- My appointments
- Session authentication
- Cookie auto-fill

6.2 Pending Features:

– Staff login

– Email notifications

– More departments

6.3 Challenges:

- Database connections
- JSP errors
- File path issues
- Session & cookie management

7. Final Tool Description

7.1 Full System Workflow:

Student → Login → Search Staff → View Profile → Book Appointment → Receive Notifications → Logout

7.2 Functional Demonstration Solutions :

- Full walkthrough of all system features
- Explanation of session and cookie handling
- Demonstration of appointment CRUD operations

8.Conclusion

In conclusion,this project provided valuable experience in full-stack development using JSP, Java, MySQL, and Tomcat.
The system implements secure session-based authentication and user-friendly cookie-based auto-fill functionality.
It successfully streamlines communication between students and instructors through search, appointments, and notifications.

Future improvements include adding staff login, email reminders, more departments , Language change , dark mode

9. References

Oracle. (2024). Java EE: JSP and Servlet documentation. Oracle Corporation. https://docs.oracle.com/javaee/7/tutorial/servlets.htm

Mozilla Developer Network. (2024). CSS Flexible Box Layout (Flexbox). MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Flexible_Box_Layout

Mozilla Developer Network. (2024). CSS Grid Layout. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Grid_Layout

Mozilla Developer Network. (2024). HTTP Cookies. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies

Oracle. (2024). Java Servlet Session Management. Oracle Corporation. https://docs.oracle.com/javaee/7/tutorial/servlets017.htm

MySQL. (2024). MySQL 8.0 Reference Manual. Oracle Corporation. https://dev.mysql.com/doc/refman/8.0/en/

10. Appendices(Code Snippets)

10.1 Session Validation Code:

Purpose: Protect pages so only logged-in users can access them.

File: index.jsp, appointment_booking.jsp

Code:

```
<%
    if (session.getAttribute("studentEmail") == null) {
        response.sendRedirect("student_login.jsp");
        return;
    }
%>
```

10.2 Cookie Creation (save email):

Purpose: Remember the student's email for auto-fill.

File: student_login.jsp

Code:

```
Cookie emailCookie = new Cookie("savedEmail", email);
emailCookie.setMaxAge(7 * 24 * 60 * 60); // 1 week
response.addCookie(emailCookie);
```

10.3 Cookie Reading(auto-fill login):

Purpose: Display saved email in login form.

File: student_login.jsp

Code:

```
String savedEmail = "";
Cookie[] cookies = request.getCookies();
if (cookies != null) {
   for (Cookie c : cookies) {
      if (c.getName().equals("savedEmail")) {
         savedEmail = c.getValue();
      }
   }
}
```

10.4 Dynamic Data Rendering (JSP Expression):

Purpose: Display database values in the UI.

File: index.jsp, staff_profile.jsp

Code:

```
<h3>Welcome, <%= studentName %></h3>
```

10.5 JSP Loop to Display Search Results:

Purpose: Render staff list dynamically.

File: department_view.jsp

Code:

```
while (rs.next()) {
%>
<div class="staff-card">
   <h3><%= rs.getString("Staff_Name") %></h3>
   <p><%= rs.getString("Email") %></p>
</div>
<%
}
```

10.6 SQL Query (Back-End Example)

Purpose: Retrieve staff by department.

File: department_view.jsp backend part

Code:

```
String sql = "SELECT * FROM Staff WHERE Dept_ID = ?";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setInt(1, deptId);
ResultSet rs = pstmt.executeQuery();
```

10.7 Appointment Insertion

Purpose: Save a new appointment in the database.

File: appointment_booking.jsp

Code:

```
String sql = "INSERT INTO Appointments (Staff_ID, Student_Name, Email, Date,
Time) VALUES (?, ?, ?, ?, ?)";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setInt(1, staffID);
pstmt.setString(2, studentName);
```

```
pstmt.setString(3, studentEmail);
pstmt.setString(4, selectedDate);
pstmt.setString(5, selectedTime);
pstmt.executeUpdate();
```

10.8 Unsubscribe from Notifications

Purpose: Remove subscription.

File: notifications.jsp

Code:

```
String sql = "DELETE FROM Notification_Subscriptions WHERE Staff_ID = ? AND
Email = ?";
PreparedStatement p = conn.prepareStatement(sql);
p.setInt(1, staffId);
p.setString(2, studentEmail);
p.executeUpdate();
```

10.9 Example CSS Layout (Flexbox)

Purpose: Centering layout elements.

File: style.css

Code:

```
.page-header {
   display: flex;
   align-items: center;
   justify-content: space-between;
}
```

10.10 CSS Grid Layout

Purpose: Split page into sidebar + content.

File: style.css

Code:

```
.container {
   display: grid;
   grid-template-columns: 250px auto;
   gap: 20px;
}
```